INTERFACE CONTROL DOCUMENT

GUI-basierter Plate Diagram Editor

Technische Schnittstellenbeschreibung aller React-Komponenten

Version 1.1 | März 2026 | Haneen Almoukdad

PROJEKT

FoPra – Plate Diagram Editor

Philipps-Universität Marburg

Department Mathematik & Informatik

DOKUMENT

Typ: Interface Control Document

Version: 1.1

Datum: März 2026

Autorin: Haneen Almoukdad

Betreuer: Prof. Dr. Jan Tünnermann

M.Sc. Teresa Dreyer

Prof. Dr. C. Bockisch

1 Einleitung

Dieses Interface Control Document (ICD) beschreibt alle Schnittstellen zwischen den React-Komponenten des Plate Diagram Editors. Es dient als technische Referenz für Entwickler und Betreuer, um die Kommunikation zwischen den Modulen nachvollziehbar zu machen.

Das Dokument umfasst:

- Das zentrale Datenmodell (Typen und Interfaces aus index.ts)
- Die Props-Schnittstellen aller UI-Komponenten
- Die Datenflüsse zwischen App.tsx und den Kindkomponenten
- Die Schnittstellen der Export-Utility (exportUtils.ts)

2 Systemarchitektur

Die Anwendung folgt einem zentralisierten State-Management-Muster. App.tsx hält den gesamten Zustand (DiagramState) und reicht Daten sowie Callbacks als Props weiter. Keine Kindkomponente verwaltet globalen Zustand.

Datei	Rolle	Beschreibung
`App.tsx`	Root	Hält DiagramState, verwaltet History (Undo/Redo), koordiniert alle Handler
`Header.tsx`	UI	Zeigt App-Titel und editierbaren Projektnamen
`Toolbar.tsx`	UI	Dateioperationen, Zoom, Undo/Redo, Export, Select, Grid-Snapping
`Sidebar.tsx`	UI	Werkzeuge zur Element-Erstellung (Nodes, Plates, Edges)
`Canvas.tsx`	Kern	SVG-Zeichenfläche: Rendering und Interaktion mit dem Diagramm
`Node.tsx`	SVG	Rendert einen einzelnen Knoten (observed, unobserved, deterministic, square)
`Edge.tsx`	SVG	Rendert eine gerichtete Kante zwischen zwei Knoten
`Plate.tsx`	SVG	Rendert einen Plate mit Drag- und Resize-Unterstützung
`NodePropertiesPanel.tsx`	UI	Bearbeitung von Label, Sampling Statement und Farbe
`StatementPanel.tsx`	UI	Anzeige und Kopieren aller Sampling Statements
`exportUtils.ts`	Utility	SVG- und PNG-Export-Funktionen
`index.ts`	Datenmodell	Zentrale Typen und Interfaces – einzige Quelle der Wahrheit

3 Zentrales Datenmodell (index.ts)

Alle Typen sind in index.ts definiert und werden projektwährend von allen Komponenten importiert.

3.1 Primitive Typen

NodeType

Feld	Typ	Beschreibung
`'observed'`	`string literal`	Beobachteter Knoten – gefüllter Kreis oder Quadrat
`'unobserved'`	`string literal`	Latenter Knoten – leerer Kreis oder Quadrat
`'deterministic'`	`string literal`	Deterministischer Knoten – doppelter Rand

NodeShape

Feld	Typ	Beschreibung
`'circle'`	`string literal`	Runde Form
`'square'`	`string literal`	Quadratische Form

ToolType

Feld	Typ	Beschreibung
`'select'`	`string literal`	Auswahl-Werkzeug
`'node-observed'`	`string literal`	Erstellt beobachteten Knoten
`'node-unobserved'`	`string literal`	Erstellt latenten Knoten
`'node-deterministic'`	`string literal`	Erstellt deterministischen Knoten
`'node-square'`	`string literal`	Erstellt quadratischen Knoten (Konstante)
`'edge'`	`string literal`	Erstellt eine Kante
`'plate'`	`string literal`	Erstellt einen Plate

3.2 Haupt-Interfaces

DiagramNode

Feld	Typ	Beschreibung
`id`	`string`	Eindeutige ID des Knotens
`x`	`number`	X-Position auf der Canvas (Pixel)
`y`	`number`	Y-Position auf der Canvas (Pixel)
`label`	`string`	Angezeigter Name des Knotens

Feld	Typ	Beschreibung
`type`	`NodeType`	Typ: observed | unobserved | deterministic
`shape`	`NodeShape`	Form: circle | square
`samplingStatement`	`string`	Probabilistische Gleichung, z. B. θ ~ Normal(μ = 0, σ = 1)
`plateId`	`string | null`	ID des umgebenden Plates (null = kein Plate)
`fillColor?`	`string | undefined`	Optionale HEX-Farbe; fehlt: Typstandard wird verwendet

DiagramEdge

Feld	Typ	Beschreibung
`id`	`string`	Eindeutige ID der Kante
`fromNodeId`	`string`	ID des Quell-Knotens
`toNodeId`	`string`	ID des Ziel-Knotens

DiagramPlate

Feld	Typ	Beschreibung
`id`	`string`	Eindeutige ID des Plates
`x`	`number`	X-Position der linken oberen Ecke
`y`	`number`	Y-Position der linken oberen Ecke
`width`	`number`	Breite des Plates in Pixeln
`height`	`number`	Höhe des Plates in Pixeln
`label`	`string`	Index-Bezeichnung, z. B. “i“, “j“, “N“
`parentPlateId`	`string | null`	ID des übergeordneten Plates (null = kein Eltern-Plate)

DiagramState

Feld	Typ	Beschreibung
`projectName`	`string`	Name des aktuellen Projekts
`nodes`	`DiagramNode[]`	Liste aller Knoten
`edges`	`DiagramEdge[]`	Liste aller Kanten
`plates`	`DiagramPlate[]`	Liste aller Plates
`selectedTool`	`ToolType`	Aktuell aktives Werkzeug
`selectedElementIds`	`string[]`	IDs der ausgewählten Elemente
`zoomLevel`	`number`	Zoom-Faktor (0.5–2.0, Standard: 1.0)

4 Komponenten-Schnittstellen (Props)

4.1 Header

Zeigt den App-Titel und editierbaren Projektnamen. Änderungen werden per Callback an App.tsx gemeldet.

Prop	Typ	Pflicht	Beschreibung
`projectName`	`string`	✓ Ja	Aktueller Projektname
`onProjectNameChange`	`(name: string) => void`	✓ Ja	Callback: Nutzer speichert neuen Projektnamen

4.2 Toolbar

Horizontale Werkzeugleiste mit Dateioperationen, Zoom, Undo/Redo und Export.

Prop	Typ	Pflicht	Beschreibung
`onNewProject`	`() => void`	✓ Ja	Neues, leeres Projekt erstellen
`onSaveAs`	`() => void`	✓ Ja	Projekt als JSON-Datei herunterladen
`onLoadProject`	`(file: File) => void`	✓ Ja	JSON-Datei laden und importieren
`onZoomIn`	`() => void`	✓ Ja	Zoom-Level um 0.1 erhöhen
`onZoomOut`	`() => void`	✓ Ja	Zoom-Level um 0.1 verringern
`onUndo`	`() => void`	✓ Ja	Letzte Aktion rückgängig machen
`onRedo`	`() => void`	✓ Ja	Rückgängig gemachte Aktion wiederherstellen
`canUndo`	`boolean`	✓ Ja	Steuert, ob Undo-Button aktiv ist
`canRedo`	`boolean`	✓ Ja	Steuert, ob Redo-Button aktiv ist
`onDelete`	`() => void`	✓ Ja	Ausgewählte Elemente löschen
`onAutoLayout`	`() => void`	✓ Ja	Auto-Layout ausführen (aktuell Platzhalter)
`onExportPng`	`() => void`	✓ Ja	Diagramm als PNG exportieren
`onExportSvg`	`() => void`	✓ Ja	Diagramm als SVG exportieren
`zoomLevel`	`number`	✓ Ja	Aktueller Zoom-Faktor (wird als % angezeigt)
`selectedTool`	`ToolType`	✓ Ja	Aktuelles Werkzeug (für Hervorhebung)
`onToolSelect`	`(tool: ToolType) => void`	✓ Ja	Werkzeug wechseln
`gridSnappingEnabled`	`boolean`	✓ Ja	Ob Grid-Snapping aktiv ist
`onToggleGridSnapping`	`() => void`	✓ Ja	Grid-Snapping ein-/ausschalten

4.3 Sidebar

Linke Seitenleiste mit den Werkzeugen zur Element-Erstellung.

Prop	Typ	Pflicht	Beschreibung
`selectedTool`	`ToolType`	✓ Ja	Aktuelles Werkzeug (für visuelle Hervorhebung)
`onToolSelect`	`(tool: ToolType) => void`	✓ Ja	Callback: Werkzeug wechseln

4.4 Canvas

SVG-Zeichenfläche. Empfängt den Diagrammzustand und meldet Benutzerinteraktionen per Callback. Hinweis: gridSnappingEnabled wird intern in App.tsx verwaltet und nicht als Canvas-Prop übergeben.

Prop	Typ	Pflicht	Beschreibung
`nodes`	`DiagramNode[]`	✓ Ja	Alle darzustellenden Knoten
`edges`	`DiagramEdge[]`	✓ Ja	Alle darzustellenden Kanten
`plates`	`DiagramPlate[]`	✓ Ja	Alle darzustellenden Plates
`selectedElementIds`	`string[]`	✓ Ja	IDs der ausgewählten Elemente
`selectedTool`	`ToolType`	✓ Ja	Aktuelles Werkzeug
`zoomLevel`	`number`	✓ Ja	Aktueller Zoom-Faktor
`onSelectElement`	`(id: string | null) => void`	✓ Ja	Element auswählen (null = aufheben)
`onUpdateNode`	`(id, updates: Partial<DiagramNode>)=>void`	✓ Ja	Knoten aktualisieren + History-Snapshot
`onUpdateNodeSilent`	`(id, updates: Partial<DiagramNode>)=>void`	✓ Ja	Knoten aktualisieren OHNE History (während Drag)
`onUpdatePlate`	`(id, updates: Partial<DiagramPlate>)=>void`	✓ Ja	Plate aktualisieren + History-Snapshot
`onUpdatePlateSilent`	`(id, updates: Partial<DiagramPlate>)=>void`	✓ Ja	Plate aktualisieren OHNE History (während Drag/Resize)
`onAddNode`	`(node: DiagramNode) => void`	✓ Ja	Neuen Knoten hinzufügen
`onAddPlate`	`(plate: DiagramPlate) => void`	✓ Ja	Neuen Plate hinzufügen
`onAddEdge`	`(edge: DiagramEdge) => void`	✓ Ja	Neue Kante hinzufügen
`onDragEnd`	`() => void`	✓ Ja	Drag beendet – History-Snapshot speichern
`svgRef`	`React.RefObject<SVGSVGElement>`	Optional	Ref auf SVG für Export; intern wird ein eigenes Ref erstellt

4.5 Node

Rendert einen einzelnen Knoten als SVG-Element innerhalb der Canvas.

Prop	Typ	Pflicht	Beschreibung
`node`	`DiagramNode`	✓ Ja	Daten des Knotens
`isSelected`	`boolean`	✓ Ja	Ob der Knoten ausgewählt ist
`isEdgeStart`	`boolean`	Optional	Startpunkt einer neuen Kante (grüne Hervorhebung). Default: false
`onSelect`	`(id: string) => void`	✓ Ja	Callback: Knoten angeklickt
`onDragStart`	`(id: string, e: React.MouseEvent) => void`	✓ Ja	Callback: Drag-Vorgang gestartet

4.6 Edge

Rendert eine gerichtete Kante als SVG-Pfeil zwischen zwei Knoten.

Prop	Typ	Pflicht	Beschreibung
`edge`	`DiagramEdge`	✓ Ja	Daten der Kante
`fromNode`	`DiagramNode`	✓ Ja	Quell-Knoten
`toNode`	`DiagramNode`	✓ Ja	Ziel-Knoten
`isSelected`	`boolean`	✓ Ja	Ob die Kante ausgewählt ist
`onSelect`	`(id: string) => void`	✓ Ja	Callback: Kante angeklickt

4.7 Plate

Rendert einen Plate als SVG-Rechteck mit Beschriftung, unterstützt Drag und Resize.

Prop	Typ	Pflicht	Beschreibung
`plate`	`DiagramPlate`	✓ Ja	Daten des Plates
`isSelected`	`boolean`	✓ Ja	Ob der Plate ausgewählt ist
`isSelectMode`	`boolean`	✓ Ja	Ob das Select-Werkzeug aktiv ist (steuert Cursor-Stil)
`depth`	`number`	✓ Ja	Verschachtelungstiefe (0 = äußerste Ebene); bestimmt Hintergrundton
`onSelect`	`(id: string) => void`	✓ Ja	Callback: Plate angeklickt
`onDragStart`	`(id: string, e: React.MouseEvent) => void`	✓ Ja	Callback: Drag gestartet

Prop	Typ	Pflicht	Beschreibung
`onResizeStart`	`(id: string, corner: string, e: React.MouseEvent)=>void`	✓ Ja	Callback: Resize an einer Ecke gestartet

4.8 NodePropertiesPanel

Rechtes Panel für Bearbeitung von Knoten- und Plate-Eigenschaften.

Prop	Typ	Pflicht	Beschreibung
`selectedNode`	`DiagramNode | null`	✓ Ja	Ausgewählter Knoten oder null
`selectedPlate`	`DiagramPlate | null`	✓ Ja	Ausgewählter Plate oder null
`onUpdateNode`	`(id, updates: Partial<DiagramNode>) => void`	✓ Ja	Knoteneigenschaften aktualisieren (Label, Statement, Farbe)
`onUpdatePlate`	`(id, updates: Partial<DiagramPlate>)=> void`	✓ Ja	Plate-Eigenschaften aktualisieren (Label/Index)

4.9 StatementPanel

Unteres Panel: Anzeige aller Sampling Statements. Nur Lesezugriff – keine Bearbeitung.

Prop	Typ	Pflicht	Beschreibung
`nodes`	`DiagramNode[]`	✓ Ja	Alle Knoten. Knoten ohne samplingStatement werden herausgefiltert.

5 Export-Schnittstelle (exportUtils.ts)

Das Modul stellt zwei asynchrone Funktionen bereit, die direkt von App.tsx aufgerufen werden.

5.1 exportAsSvg()

Prop	Typ	Pflicht	Beschreibung
`svgElement`	`SVGSVGElement`	✓ Ja	Referenz auf das SVG-DOM-Element der Canvas
`nodes`	`DiagramNode[]`	✓ Ja	Knotenliste (für Bounding-Box-Berechnung)
`edges`	`DiagramEdge[]`	✓ Ja	Kantenliste
`plates`	`DiagramPlate[]`	✓ Ja	Plateliste
`projectName`	`string`	✓ Ja	Dateiname für den Download (ohne Endung)

5.2 exportAsPng()

Identische Signatur wie exportAsSvg(). Konvertiert das SVG intern über die Canvas-API in ein PNG-Raster und löst den Download aus.

Prop	Typ	Pflicht	Beschreibung
`svgElement`	`SVGSVGElement`	✓ Ja	Referenz auf das SVG-DOM-Element der Canvas
`nodes`	`DiagramNode[]`	✓ Ja	Knotenliste
`edges`	`DiagramEdge[]`	✓ Ja	Kantenliste
`plates`	`DiagramPlate[]`	✓ Ja	Plateliste
`projectName`	`string`	✓ Ja	Dateiname für den Download (ohne Endung)

6 Datenflüsse

6.1 Zustandsfluss — App.tsx → Kindkomponenten

Komponente	Übergebene Daten
`Header`	projectName
`Toolbar`	zoomLevel, selectedTool, canUndo, canRedo, gridSnappingEnabled
`Sidebar`	selectedTool
`Canvas`	nodes, edges, plates, selectedTool, selectedElementIds, zoomLevel, svgRef
`NodePropertiesPanel`	selectedNode (aus nodes), selectedPlate (aus plates)
`StatementPanel`	nodes

6.2 Ereignisfluss — Kindkomponenten → App.tsx

Callback	Ausgelöste Aktion
`onProjectNameChange`	Projektname in DiagramState aktualisieren
`onToolSelect`	selectedTool ändern
`onUndo / onRedo`	History-Pointer verschieben, DiagramState wiederherstellen
`onExportPng / onExportSvg`	exportUtils-Funktion aufrufen
`onAddNode`	Neuen Knoten zu nodes hinzufügen + History-Snapshot
`onSelectElement`	selectedElementIds aktualisieren

Callback	Ausgelöste Aktion
`onDragEnd`	History-Snapshot speichern
`onUpdateNode (Panel)`	Knoteneigenschaften ändern + History-Snapshot
`onUpdatePlate (Panel)`	Plate-Label ändern + History-Snapshot

7 Speicherformat (JSON)

Beim Speichern wird DiagramState als JSON-Datei serialisiert. Beim Laden wird validiert: nodes, edges und plates müssen als Arrays vorhanden sein, sonst erscheint eine Fehlermeldung.

Feld	Typ	Beschreibung
`projectName`	`string`	Projektname
`nodes`	`DiagramNode[]`	Alle Knoten mit vollständigen Eigenschaften
`edges`	`DiagramEdge[]`	Alle Kanten
`plates`	`DiagramPlate[]`	Alle Plates
`selectedTool`	`ToolType`	Zuletzt aktives Werkzeug
`selectedElementIds`	`string[]`	Zuletzt ausgewählte Elemente
`zoomLevel`	`number`	Letzter Zoom-Faktor